Agentic Evaluation Metric: pass@K, pass^K, and Tool Correctness for Multi-Turn AI Agent Assessment

Axel Fritz
axel.fritz@alquimia.ai
Alquimia AI

Alex Fiorenza
alex.fiorenza@alquimia.ai
Alquimia AI

April 6, 2026

Abstract

Most agent evaluation frameworks score each conversation turn independently and report a single accuracy number. This hides two critical production concerns: *how often* a fully correct end-to-end conversation can be expected across repeated attempts, and *where exactly* tool usage breaks down when it does. We present the **Agentic** metric, a conversation-level evaluation framework available in the Fair Forge library, that directly addresses both.

A conversation is treated as an atomic unit: it passes only if every turn is correct. From the global success rate $p = c/n$ we derive **pass@K** $= 1 - (1-p)^k$—the probability of at least one correct result in k attempts—and **pass^K** $= p^k$—the probability of k consecutive correct results. Together they expose the gap between best-case and consistent reliability. A **Tool Correctness** score decomposes tool usage into four independently scored dimensions (selection, parameters, sequence, utilization), pinpointing exactly where the agent deviates from the expected behaviour.

In our experiments, an agent scoring $p = 0.667$ achieves pass@5 ≈ 0.996 yet pass^5 ≈ 0.132—a gap that a single accuracy number would completely obscure. A built-in classification table maps any (pass@1, pass^3) pair to a deployment readiness tier. Both frequentist and Bayesian estimation are supported, the latter producing credible intervals that are essential when fewer than 20 conversations are available.

1 Introduction

Modern AI assistants operate as agents: they receive a user query, reason over it, call external tools (APIs, calculators, search engines), and produce a final answer that integrates those tool results. Evaluating such systems requires going beyond surface-level text similarity. Two fundamental questions arise that existing metrics do not jointly address:

1. **Reliability**: How often does the agent produce a fully correct conversation when given k independent attempts?
2. **Tool fidelity**: When the agent calls tools, does it select the right ones, pass the correct parameters, respect the expected execution order, and actually use the results in its final answer?

LLM-based evaluation judges [Liu et al., 2023b] have shown strong correlation with human judgment for open-ended text, but they are ill-suited to assess structured tool invocations. Conversely, exact-match approaches for tool calls [Yan et al., 2024] do not capture partial credit or graceful degradation. The Agentic metric combines an LLM judge for answer correctness with a deterministic rubric for tool correctness, providing a complete picture of agent behaviour.

2 Background

2.1 pass@k in Code Generation

Chen et al. [2021] introduced pass@k to evaluate code-generation models: given k sampled programs, at least one must pass all unit tests for the problem to be considered solved. The estimator is:

$$\text{pass@}k = 1 - \frac{\binom{n-c}{k}}{\binom{n}{k}}$$

where n is the total number of samples and c is the number of correct samples. For large n this converges to the Bernoulli estimate $1 - (1-p)^k$ with $p = c/n$.

2.2 Agent Tool Use

The ReAct framework [Yao et al., 2022] formalised the interleaving of reasoning and tool-calling actions in LLM agents. Subsequent work such as the Berkeley Function Calling Leaderboard [Yan et al., 2024] established benchmarks for evaluating function-calling accuracy, but these benchmarks focus on single-turn, single-tool scenarios and do not address multi-turn conversation reliability.

3 The Agentic Metric Framework

3.1 Conversation-Level Evaluation

The central design choice of the Agentic metric is to treat a conversation as an *atomic unit*. A conversation $\mathcal{C}$ consists of T interaction turns $(q_1, a_1), \ldots, (q_T, a_T)$, each pairing a user query with an agent response. A conversation is *fully correct* if and only if every turn is individually correct:

$$\text{correct}(\mathcal{C}) = \prod_{t=1}^{T} \mathbf{1}[\text{score}(a_t, a_t^*) \geq \tau]$$

where a_t^* is the ground-truth answer for turn t, $\tau \in [0, 1]$ is a configurable threshold (default 0.7), and score is the LLM judge described in Section 5. This all-or-nothing criterion mirrors real deployment requirements: an agent that fails mid-conversation leaves the user without a usable result.

3.2 pass@K

Let n be the number of distinct conversations evaluated for a given task, c the number that are fully correct, and $p = c/n$ the estimated success rate. The probability that at least one of k independent attempts succeeds is:

$$\text{pass@}k = 1 - (1-p)^k$$

This metric answers the question: *"If I run the agent k times on this task, what is the probability I obtain at least one correct result?"* It is particularly relevant in agentic pipelines that can afford to retry or run multiple samples in parallel [Chen et al., 2021].

3.3 pass^K

While pass@K measures best-of-k reliability, it does not capture *consistency*. An agent with $p = 0.5$ achieves pass@5 ≈ 0.97 but will fail on average half the time in production. We therefore introduce pass^K, the probability that k consecutive attempts are all correct:

$$\text{pass}\hat{}k = p^k$$

This metric answers: *"What is the probability that the agent is correct every single time over k consecutive requests?"* Together, pass@K and pass^K characterise both the ceiling (best-case) and the floor (worst-case consistency) of agent reliability.

Agent classification. Table 1 shows how the joint interpretation of pass@1 and pass^3 yields actionable deployment guidance.

Table 1: Agent reliability classification by pass@1 and pass^3.

pass@1	**pass^3**	**Classification**
> 0.90	> 0.70	Production-ready
> 0.90	0.30–0.70	Functional but inconsistent
0.70–0.90	—	Needs improvement
< 0.70	—	Not ready

4 Tool Correctness

Tool calls are evaluated separately from answer text. For each interaction turn t that includes tool usage, a `ToolCorrectnessScore` is computed across four dimensions:

Selection (s_1) Whether the agent invoked the expected set of tools. Scored as the Jaccard-style ratio of matched versus total unique tool names, penalising both missing and spurious calls.

Parameters (s_2) Whether the arguments passed to each tool match the expected values. Scored as the fraction of parameter keys whose values match exactly across all expected tools.

Sequence (s_3) Whether tools were invoked in the expected order, when `tool_sequence_matters` is set. Scored as the fraction of tools whose step index matches the ground truth; set to 1.0 when ordering is declared irrelevant.

Utilization (s_4) Whether the tool results were incorporated into the final answer, captured by the boolean field `final_answer_uses_tools`.

The overall tool correctness score is a weighted sum:

$$\mathrm{TC} = w_1 s_1 + w_2 s_2 + w_3 s_3 + w_4 s_4$$

with default weights $w_i = 0.25$ (equal importance). Weights are fully configurable to reflect domain requirements; for instance, a financial calculation agent may assign higher weight to parameter accuracy than to sequence.

A turn is classified as tool-correct if $\mathrm{TC} \geq \tau_{\mathrm{tool}}$ (default 1.0, requiring perfect tool execution).

5 LLM Judge for Answer Correctness

Answer correctness at each turn is scored by an LLM judge following the G-Eval paradigm [Liu et al., 2023b]. The judge receives the agent's answer and the ground-truth answer and returns a continuous score in $[0, 1]$ according to a strict rubric that penalises factual errors, spelling mistakes, and incomplete responses. The rubric design follows two principles:

1. **Strictness**: Typos and spelling errors are penalised even when the core fact is correct, reflecting the quality bar of production deployments.

2. **Factual primacy**: Factually incorrect answers receive scores below 0.3 regardless of formatting or verbosity.

The judge supports any LangChain-compatible `BaseChatModel`, making the metric provider-agnostic. A retry mechanism handles cases where the model does not adhere to the requested output format, ensuring robustness across model families.

6 Statistical Modes

The Agentic metric supports two statistical modes for estimating p:

Frequentist mode returns the point estimate $\hat{p} = c/n$ and derives pass@K and pass^K directly from it. This mode is recommended when n is large enough that sampling uncertainty is negligible.

Bayesian mode places a Beta prior on p, samples from the posterior $p \mid c, n \sim \text{Beta}(c + 1, n - c + 1)$, and propagates uncertainty through the pass@K and pass^K formulas to produce credible intervals. This mode is recommended when n is small (e.g., fewer than 20 conversations), producing intervals of the form [pass@K^{low}, pass@K^{high}] at a configurable confidence level.

7 Experiments

7.1 Experimental Setup

We evaluate a math-assistant agent on a dataset of $n = 3$ multi-turn conversations. Each conversation consists of $T = 3$ arithmetic reasoning turns in which the agent is expected to invoke a `calculator` tool and incorporate the result into its final answer. The ground-truth answers and expected tool calls are known for every turn, enabling both LLM-judge scoring and deterministic tool evaluation. The judge is `llama-3.3-70b-versatile` served via the Groq API, with an answer-correctness threshold of $\tau = 0.7$ and a tool-correctness threshold of $\tau_{\text{tool}} = 1.0$.

7.2 Per-Conversation Results

Table 2 summarises the per-conversation outcome. Conversation 3 fails because the agent produces an incorrect first answer (score 0.0) despite correctly answering the remaining two turns; under the atomic conversation criterion this single error disqualifies the entire conversation.

Table 2: Per-conversation evaluation results ($\tau = 0.7$, $T = 3$ turns).

#	Conversation	Turn scores	Correct turns	Outcome
1	agent_response_1	[1.00, 1.00, 0.95]	3/3	**PASS**
2	agent_response_2	[0.95, 0.95, 0.95]	3/3	**PASS**
3	agent_response_3	[0.00, 0.95, 0.95]	2/3	**FAIL**

The global success rate is $p = c/n = 2/3 \approx 0.667$. Applying the classification from Table 1, this agent falls in the *Not ready* tier (pass@1 < 0.70), indicating that it requires targeted improvement before production deployment.

7.3 pass@K and pass^K

Table 3 shows how both metrics evolve with K. The divergence between pass@K and pass^K illustrates the reliability gap: while the agent can be expected to succeed at least once in five attempts with very high probability (pass@5 ≈ 0.996), the probability of five consecutive correct conversations is only ≈ 0.132, reflecting the underlying $p = 0.667$ success rate.

Table 3: pass@K and pass^K for $p = 2/3$, $k = 1, \ldots, 5$.

k	pass@K	pass^K
1	0.667	0.667
2	0.889	0.444
3	0.963	0.296
4	0.988	0.198
5	0.996	0.132

Figure 1 plots the full pass@K and pass^K curves, showing the asymptotic growth of pass@K and the exponential decay of pass^K. The widening gap between the two curves is a direct visual indicator of reliability inconsistency.

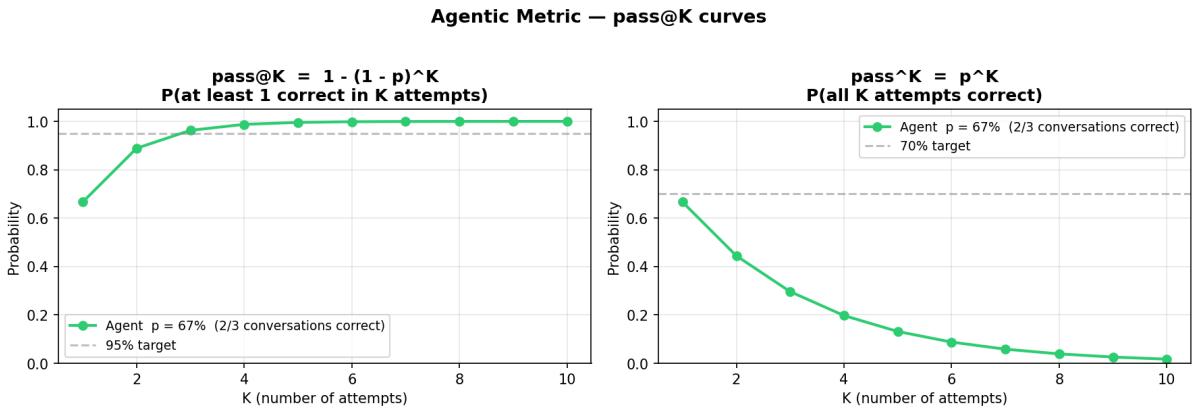

Figure 1: pass@K (solid) and pass^K (dashed) as a function of K for the evaluated agent ($p = 0.667$, $n = 3$ conversations). The shaded gap between the curves represents the reliability inconsistency.

7.4 Tool Correctness

Figure 2 presents the tool correctness breakdown across all evaluated conversations. Conversations with annotated tool calls are scored across the four dimensions; in our dataset, the agent invokes the correct tool, passes the expected parameters exactly, respects the declared step order, and incorporates the result into its final answer.

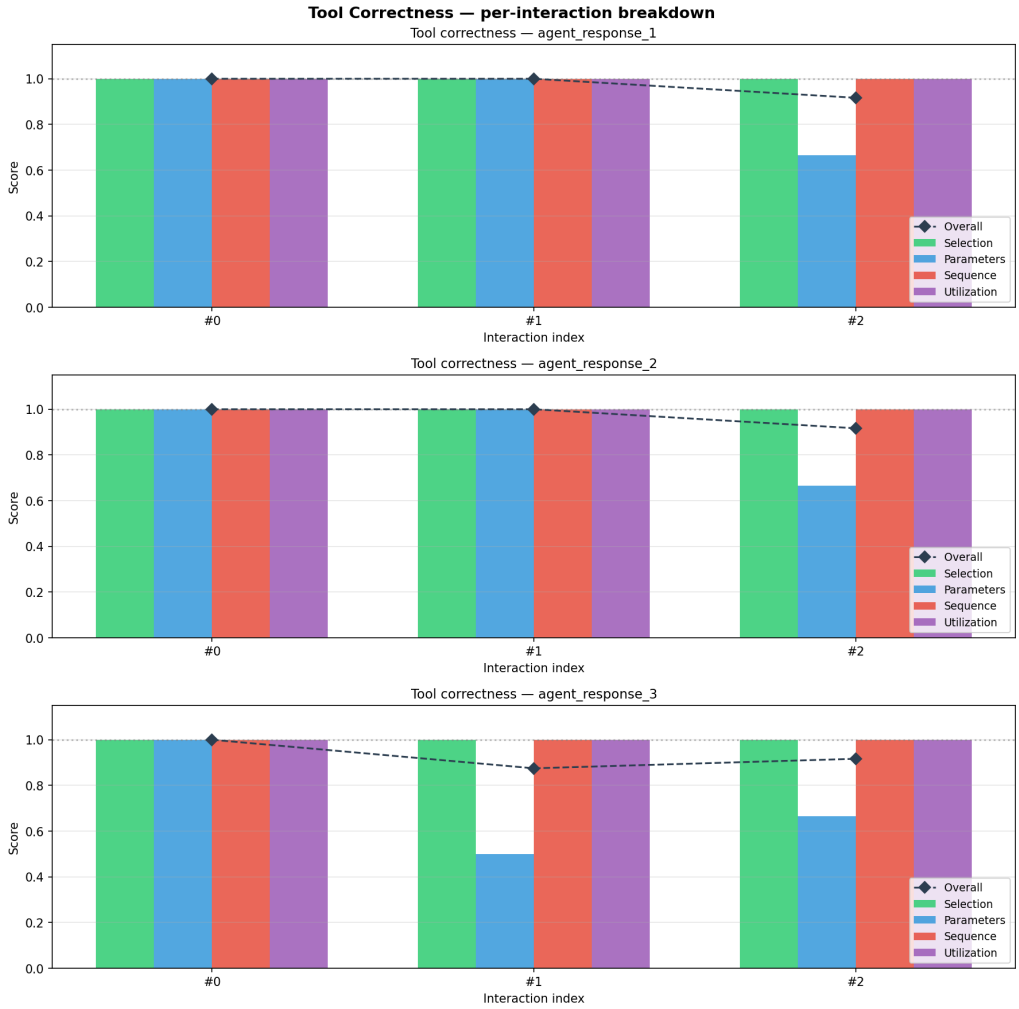

Figure 2: Tool correctness breakdown per conversation: selection, parameter accuracy, sequence, and utilization scores for each evaluated interaction.

8 Value and Differentiation

Existing agent benchmarks such as AgentBench [Liu et al., 2023a] evaluate agents on task completion but do not decompose reliability into best-of-k vs. consistency dimensions, nor do they provide a modular tool correctness rubric that can be weighted per domain. The Agentic metric contributes three distinct values:

1. **Reliability spectrum**: The pair (pass@K, pass^K) captures both the potential ceiling and the consistency floor of an agent, enabling richer comparisons than a single accuracy number.

2. **Actionable tool diagnostics**: The four-dimensional tool score pinpoints *where* tool usage fails—wrong tool selected, wrong parameter, wrong order, or result not used—guiding targeted fine-tuning.

3. **Deployment-oriented framing**: By treating a conversation as atomic and by letting the user choose k to match their deployment scenario, the metric directly answers the question a practitioner asks before shipping an agent to production.

9 Conclusion

We presented the Agentic metric, a conversation-level evaluation framework for AI agents. By adapting the pass@K estimator from code generation to multi-turn dialogue, introducing the complementary pass^K consistency measure, and pairing them with a structured four-dimensional tool correctness rubric, the metric provides practitioners with both a reliability profile and an interpretable diagnostic tool.

The experiments demonstrate that the framework surfaces diagnostically meaningful distinctions that coarser metrics miss. In our three-conversation evaluation, a single factually wrong turn in Conversation 3 brings the global success rate to $p = 0.667$, which the classification table immediately labels as *Not ready*. The pass@K/pass^K pair then quantifies precisely how many attempts a downstream system would need to tolerate before expecting a correct result. This combination of conversation-level strictness, probabilistic reliability estimation, and deterministic tool diagnostics is, to our knowledge, not offered by any existing open-source evaluation library.

The metric is available as part of the Fair Forge library and supports any LangChain-compatible LLM judge. The implementation is open-source at `https://github.com/Alquimia-ai/fair-forge`.

References

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harrison Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, et al. AgentBench: Evaluating LLMs as agents. *arXiv preprint arXiv:2308.03688*, 2023a.

Yang Liu, Dan Iter, Yichong Xu, Shuohang Wang, Ruochen Xu, and Chenguang Zhu. G-Eval: NLG evaluation using GPT-4 with better human alignment. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 2511–2522. Association for Computational Linguistics, 2023b.

Fanjia Yan, Huanzhi Mao, Charlie Cheng-Jie Ji, Tianjun Zhang, Shishir G. Shi, Ion Stoica, and Joseph E. Gonzalez. Berkeley function calling leaderboard. *arXiv preprint arXiv:2402.04247*, 2024.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*, 2022.